Session 08 - 2D Arrays, Array Reverse, Objects, References

Naveen AutomationLabs | 2026 Playwright Batch G1

---

1. Two-Dimensional (2D) Arrays

A 2D array is simply an array that contains other arrays as its elements. Think of it as rows and columns, like a table.

```
let matrix = [
[1, 2, 3],
[4, 5, 6],
[7, 8, 9]
];
console.log(matrix[0][0]); // 1
console.log(matrix[1][2]); // 6
console.log(matrix[2][1]); // 8
```

To loop through all elements, use nested for loops:

```
for (let i = 0; i < matrix.length; i++) {
for (let j = 0; j < matrix[i].length; j++) {
console.log(matrix[i][j]);
}
}
```

matrix.length gives the number of rows. matrix[i].length gives the number of columns in that row.

2. Array Reverse

The reverse() method reverses the array in place. It changes the original array.

```
let nums = [10, 20, 30, 40, 50];
nums.reverse();
console.log(nums); // [50, 40, 30, 20, 10]
```

reverse() modifies the original array. If you want to keep the original, make a copy first using slice() or the spread operator.

3. Array Concatenation with + Operator

When you use the + operator between arrays, JavaScript converts them to strings first and then concatenates.

```
let x = [1, 2, 3];
let y = [4, 5];
console.log(x + y); // "1,2,34,5" (string, not array)
```

This is a common gotcha. To merge arrays properly, use concat() or the spread operator: [...x, ...y].

4. Objects - Basics

An object stores data in key-value pairs. Keys are strings (property names), values can be any type.

```
let user = {
name: 'Tom',
age: 30,
city: 'LA',
isActive: true
};

// Access properties:
console.log(user.name); // dot notation
console.log(user['city']); // bracket notation

// Add a new property:
user.email = 'tom@gmail.com';

// Delete a property:
delete user.age;

// Update a property:
user.name = 'Tom Automation';
console.log(user);
```

5. Nested Objects

An object can have another object as a property value. This is called nesting.

```
let user = {
name: 'Tom',
age: 30,
address: {
flat: 101,
zip: 545454,
lat: 10.34,
long: 11.22
}
};

console.log(user.address.flat); // 101
console.log(user.address['zip']); // 545454
console.log(user['address']['flat']); // 101
```

6. Object References

When you assign one object variable to another, they both point to the same object in memory. No copy is created.

```
let u1 = { name: 'ravi', age: 30, city: 'LA' };
let u2 = { name: 'ankita', age: 20, city: 'pune' };

u1 = u2;
console.log(u1); // { name: 'ankita', age: 20, city: 'pune' }
console.log(u2); // { name: 'ankita', age: 20, city: 'pune' }

// Both u1 and u2 now point to the same object.
// The original u1 object { name: 'ravi' } is lost.
```

This is called 'reference assignment'. Changing u1.name will also change u2.name because both point to the same object.